

Binary Codes

In the coding, when numbers, letters or words are represented by a specific group of symbols, it is said that the number, letter or word is being encoded. The group of symbols is called as a code. The digital data is represented, stored and transmitted as group of binary bits. This group is also called as **binary code**. The binary code is represented by the number as well as alphanumeric letter.

Advantages of Binary Code

Following is the list of advantages that binary code offers.

- Binary codes are suitable for the computer applications.
- Binary codes are suitable for the digital communications.
- Binary codes make the analysis and designing of digital circuits if we use the binary codes.
- Since only 0 & 1 are being used, implementation becomes easy.

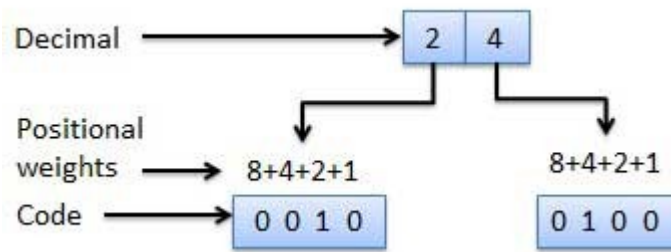
Classification of binary codes

The codes are broadly categorized into following four categories.

- Weighted Codes
- Non-Weighted Codes
- Binary Coded Decimal Code
- Alphanumeric Codes
- Error Detecting Codes
- Error Correcting Codes

Weighted Codes

Weighted binary codes are those binary codes which obey the positional weight principle. Each position of the number represents a specific weight. Several systems of the codes are used to express the decimal digits 0 through 9. In these codes each decimal digit is represented by a group of four bits.

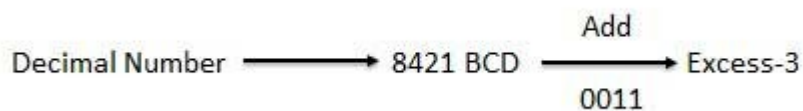


Non-Weighted Codes

In this type of binary codes, the positional weights are not assigned. The examples of non-weighted codes are **Excess-3 code** and **Gray code**.

Excess-3 code

The Excess-3 code is also called as XS-3 code. It is non-weighted code used to express decimal numbers. The Excess-3 code words are derived from the 8421 BCD code words adding (0011)₂ or (3)₁₀ to each code word in 8421. The excess-3 codes are obtained as follows –



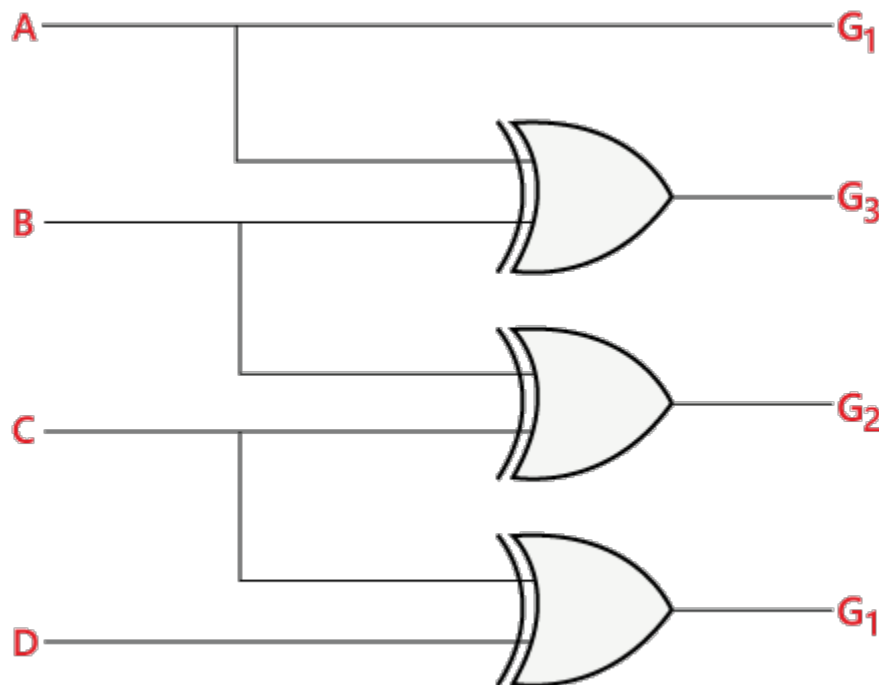
Gray Code

It is the non-weighted code and it is not arithmetic codes. That means there are no specific weights assigned to the bit position. It has a very special feature that, only one bit will change each time the decimal number is incremented as shown in fig. As only one bit changes at a time, the gray code is called as a unit distance code. The gray code is a cyclic code. Gray code cannot be used for arithmetic operation.

The 4-bit binary to gray code conversion table is as follows:

Decimal Number	4-bit Binary Code	4-bit Gray Code
	ABCD	G₁G₂G₃G₄
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011

14	1110	1001
15	1111	1000



Logic Circuit for Binary to Gray Code Converter

How to Convert Binary to Gray Code

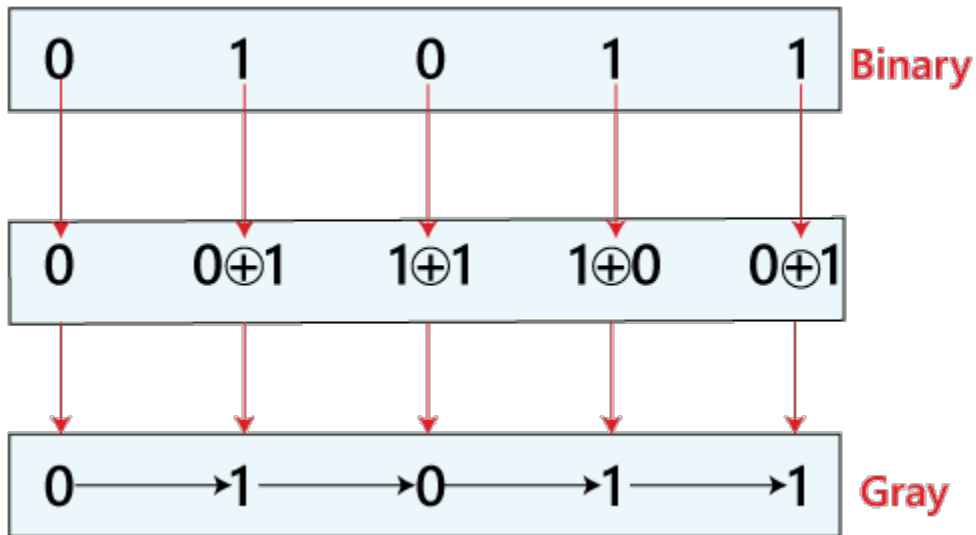
- o In the Gray code, the MSB will always be the same as the 1st bit of the given binary number.
- o In order to perform the 2nd bit of the gray code, we perform the exclusive-or (XOR) of the 1st and 2nd bit of the binary number. It means that if both the bits are different, the result will be one else the result will be 0.
- o In order to get the 3rd bit of the gray code, we need to perform the exclusive-or (XOR) of the 2nd and 3rd bit of the binary number. The process remains the same for the 4th bit of the Gray code. Let's take an example to understand these steps.

Example

Suppose we have a binary number 01101, which we want to convert into Gray code. There are the following steps which need to perform this conversion:

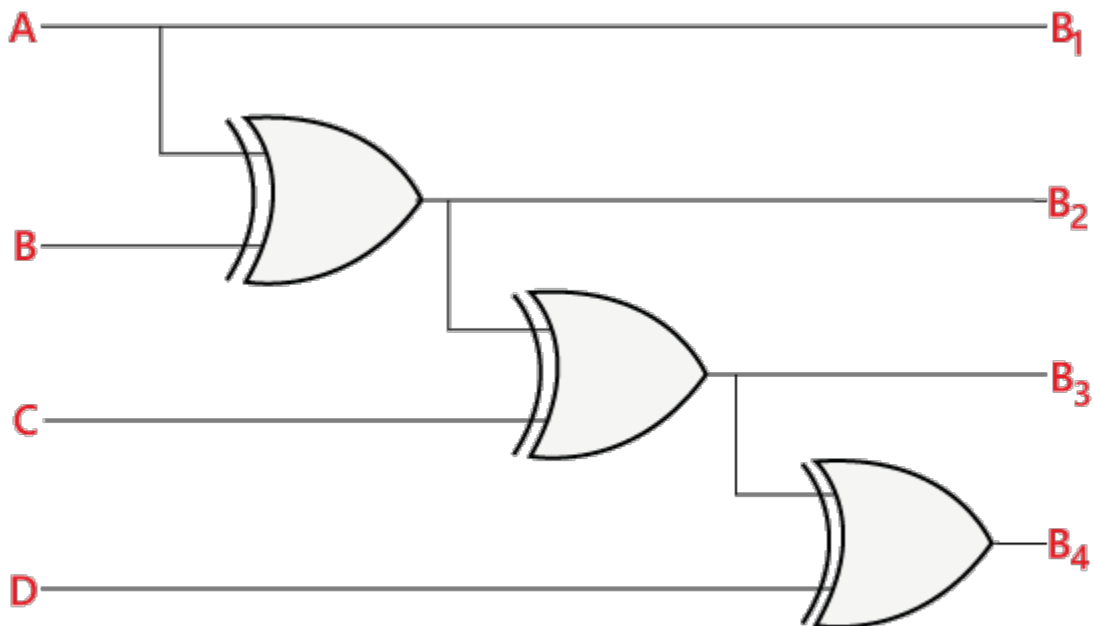
ADVERTISEMENT

- o As we know that the 1st bit of the Gray code is the same as the MSB of the binary number. In our example, the MSB is 0, so the MSB or 1st bit of the gray code is 0.
- o Next, we perform the XOR operation of the 1st and the second binary number. The 1st bit is 0, and the 2nd bit is 1. Both the bits are different, so the 2nd bit of the Gray code is 1.
- o Now, we perform the XOR of the 2nd bit and 3rd bit of the binary number. The 2nd bit is 1, and the 3rd bit is also 1. These bits are the same, so the 3rd bit of the Gray code is 0.
- o Again perform the XOR operation of the 3rd and 4th bit of binary number. The 3rd bit is 1, and the 4th bit is 0. As these are different, the 4th bit of the Gray code is 1.
- o Lastly, perform the XOR of the 4th bit and 5th bit of the binary number. The 4th bit is 0, and the 5th bit is 1. Both the bits are different, so that the 5th bit of the Gray code is 1.
- o The gray code of the binary number 01101 is 01011.



Gray to Binary Code Conversion

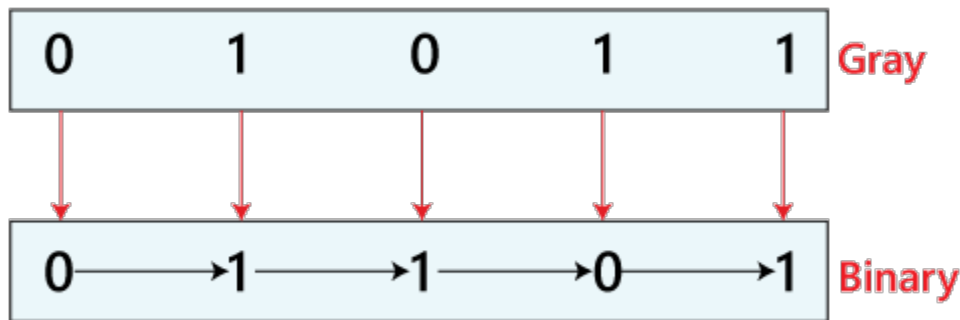
The Gray to Binary code converter is a logical circuit that is used to convert the gray code into its equivalent binary code. There is the following circuit used to convert the Gray code to binary number.



Logic Circuit for Gray to Binary Code Converter

Just like Binary to Gray code conversion; it is also a very simple process. There are the following steps used to convert the Gray code into binary.

- o Just like binary to gray, in gray to binary, the 1st bit of the binary number is similar to the MSB of the Gray code.
- o The 2nd bit of the binary number is the same as the 1st bit of the binary number when the 2nd bit of the Gray code is 0; otherwise, the 2nd bit is altered bit of the 1st bit of binary number. It means if the 1st bit of the binary is 1, then the 2nd bit is 0, and if it is 0, then the 2nd bit be 1.
- o The 2nd step continues for all the bits of the binary number.



Gray Code to Binary Conversion Example

Suppose we have the Gray code 01011, which we want to convert into a binary number. There are the following steps which we need to perform for the conversion:

- o The 1st bit of the binary number is the same as the MSB of the Gray code. The MSB of the Gray code is 0, so the MSB of the binary number is 0.
- o Now, for the 2nd bit, we check the 2nd bit of the Gray code. The 2nd bit of the Gray code is 1, so the 2nd bit of the binary number is one that is altered number of 1st
- o The next bit of the Gray code is 0; the 3rd bit is the same as the 2nd bit of the Gray code, i.e., 1.
- o The 4th bit of the Gray code is 1; the 4th bit of the binary number is 0 that is the altered number of the 3rd

- o The 5th bit of the Gray code is 1; the 5th bit of the binary number is 1; that is the altered number of the 4th bit of the binary number.
- o So, the binary number of the Gray code 01011 is 01101.

Application of Gray code

- Gray code is popularly used in the shaft position encoders.
- A shaft position encoder produces a code word which represents the angular position of the shaft.

Binary Coded Decimal (BCD) code

In this code each decimal digit is represented by a 4-bit binary number. BCD is a way to express each of the decimal digits with a binary code. In the BCD, with four bits we can represent sixteen numbers (0000 to 1111). But in BCD code only first ten of these are used (0000 to 1001). The remaining six code combinations i.e. 1010 to 1111 are invalid in BCD.

Decimal	0	1	2	3	4	5	6	7	8	9
BCD	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001

Advantages of BCD Codes

- It is very similar to decimal system.
- We need to remember binary equivalent of decimal numbers 0 to 9 only.

Disadvantages of BCD Codes

- The addition and subtraction of BCD have different rules.
- The BCD arithmetic is little more complicated.
- BCD needs more number of bits than binary to represent the decimal number. So BCD is less efficient than binary.

Alphanumeric codes

A binary digit or bit can represent only two symbols as it has only two states '0' or '1'. But this is not enough for communication between two computers because there we need many more

symbols for communication. These symbols are required to represent 26 alphabets with capital and small letters, numbers from 0 to 9, punctuation marks and other symbols.

The alphanumeric codes are the codes that represent numbers and alphabetic characters. Mostly such codes also represent other characters such as symbol and various instructions necessary for conveying information. An alphanumeric code should at least represent 10 digits and 26 letters of alphabet i.e. total 36 items. The following three alphanumeric codes are very commonly used for the data representation.

- American Standard Code for Information Interchange (ASCII).
- Extended Binary Coded Decimal Interchange Code (EBCDIC).
- Five bit Baudot Code.

ASCII code is a 7-bit code whereas EBCDIC is an 8-bit code. ASCII code is more commonly used worldwide while EBCDIC is used primarily in large IBM computers.

Error Codes

There are binary code techniques available to detect and correct data during data transmission.

Binary Arithmetic

Binary arithmetic is essential part of all the digital computers and many other digital system.

Binary Addition

It is a key for binary subtraction, multiplication, division. There are four rules of binary addition.

Case	A	+	B	Sum	Carry
1	0	+	0	0	0
2	0	+	1	1	0
3	1	+	0	1	0
4	1	+	1	0	1

In fourth case, a binary addition is creating a sum of ($1 + 1 = 10$) i.e. 0 is written in the given column and a carry of 1 over to the next column.

Example – Addition

$$\begin{array}{r} 0011010 + 001100 = 00100110 \\ \begin{array}{r} 11 \quad \text{carry} \\ 0011010 = 26_{10} \\ + 0001100 = 12_{10} \\ \hline 0100110 = 38_{10} \end{array} \end{array}$$

Binary Subtraction

Subtraction and Borrow, these two words will be used very frequently for the binary subtraction. There are four rules of binary subtraction.

Example – Subtraction

$$\begin{array}{r} 0011010 - 001100 = 00001110 \\ \begin{array}{r} 11 \quad \text{borrow} \\ 0011010 = 26_{10} \\ - 0001100 = 12_{10} \\ \hline 0001110 = 14_{10} \end{array} \end{array}$$

Binary Multiplication

Binary multiplication is similar to decimal multiplication. It is simpler than decimal multiplication because only 0s and 1s are involved. There are four rules of binary multiplication.

Case	A	x	B	Multiplication
1	0	x	0	0
2	0	x	1	0
3	1	x	0	0
4	1	x	1	1

Example – Multiplication

Example:

$$0011010 \times 001100 = 100111000$$

$$\begin{array}{r} 0011010 = 26_{10} \\ \times 0001100 = 12_{10} \\ \hline 0000000 \\ 0000000 \\ 0011010 \\ 0011010 \\ \hline 0100111000 = 312_{10} \end{array}$$

1's and 2's complement of a Binary Number

1's complement of a binary number is another binary number obtained by toggling all bits in it, i.e., transforming the 0 bit to 1 and the 1 bit to 0.

$$+7 = 00000111 \text{ equivalent } -7$$

$$00000111 \text{ 1's complement } = 11111000$$

$$\underline{11111000 \text{ 2's complement}}$$

$$11111000$$

$$\begin{array}{r} 11111000 \\ +1 \\ \hline 11111001 \end{array}$$

$$+16 + (-32) = +16 - 32$$

$$+16 = 00010000$$

$$\underline{-32 = 11100000}$$

$$+32 = 00100000$$

$$= 11011111 (1's \text{ complement})$$

$$\begin{array}{r} 11011111 \\ +1 \\ \hline -32 = 11100000 \end{array}$$

Examples:

1's complement of "0111" is "1000"

1's complement of "1100" is "0011"

2's complement of a binary number is 1 added to the 1's complement of the binary number.
Examples:

2's complement of "0111" is "1001"

2's complement of "1100" is "0100"

Boolean algebra

The logical symbol 0 and 1 are used for representing the digital input or output. The symbols "1" and "0" can also be used for a permanently open and closed digital circuit. The digital circuit can be made up of several logic gates. To perform the logical operation with minimum logic gates, a set of rules were invented, known as the **Laws of Boolean Algebra**. These rules are used to reduce the number of logic gates for performing logic operations.

The Boolean algebra is mainly used for simplifying and analyzing the complex Boolean expression. It is also known as **Binary algebra** because we only use binary numbers in this. **George Boole** developed the binary algebra in **1854**.

Rules in Boolean algebra

1. Only two values(1 for high and 0 for low) are possible for the variable used in Boolean algebra.
2. The overbar(-) is used for representing the complement variable. So, the complement of variable C is represented as $\bar{C}$.
3. The plus(+) operator is used to represent the ORing of the variables.
4. The dot(.) operator is used to represent the ANDing of the variables.

Properties of Boolean algebra

These are the following properties of Boolean algebra:

Laws and Theorems of Boolean Algebra

Operations with 0 and 1:

$$1. \quad X + 0 = X$$

$$1D. \quad X \cdot 1 = X$$

$$2. \quad X + 1 = 1$$

$$2D. \quad X \cdot 0 = 0$$

Idempotent laws

$$3. \quad X + X = X$$

$$3D. \quad X \cdot X = X$$

Involution law:

$$4. \quad (X')' = X$$

Laws of complementarity:

$$5. \quad X + X' = 1$$

$$5D. \quad X \cdot X' = 0$$

Commutative laws:

$$6. \quad X + Y = Y + X$$

$$6D. \quad X \cdot Y = Y \cdot X$$

Associative laws:

$$7. \quad (X + Y) + Z = X + (Y + Z) \\ = X + Y + Z$$

$$7D. \quad (XY)Z = X(YZ) = XYZ$$

Distributive laws:

$$8. \quad X(Y + Z) = XY + XZ$$

$$8D. \quad X + YZ = (X + Y)(X + Z)$$

Simplification theorems:

$$9. \quad XY + XY' = X$$

$$9D. \quad (X + Y)(X + Y') = X$$

$$10. \quad X + XY = X$$

$$10D. \quad X(X + Y) = X$$

$$11. \quad (X + Y')Y = XY$$

$$11D. \quad XY' + Y = X + Y$$

DeMorgan's laws:

$$12. \quad (X + Y + Z + \dots)' = X'Y'Z' \dots \quad 12D. \quad (XYZ \dots)' = X' + Y' + Z' + \dots$$

$$13. \quad [f(X_1, X_2, \dots, X_N, 0, 1, +, \cdot)]' = f(X_1', X_2', \dots, X_N', 1, 0, \cdot, +)$$

Duality:

$$14. \quad (X + Y + Z + \dots)^D = XYZ \dots \quad 14D. \quad (XYZ \dots)^D = X + Y + Z + \dots$$

$$15. \quad [f(X_1, X_2, \dots, X_N, 0, 1, +, \cdot)]^D = f(X_1, X_2, \dots, X_N, 1, 0, \cdot, +)$$

Theorem for multiplying out and factoring:

$$16. \quad (X + Y)(X' + Z) = XZ + X'Y \quad 16D. \quad XY + X'Z = (X + Z)(X' + Y)$$

Consensus theorem:

$$17. \quad XY + YZ + X'Z = XY + X'Z \quad 17D. \quad (X + Y)(Y + Z)(X' + Z) \\ = (X + Y)(X' + Z)$$

Annulment Law

When the variable is AND with 0, it will give the result 0, and when the variable is OR with 1, it will give the result 1, i.e.,

$$A.0 = 0$$

$$A+1 = 1$$

Identity Law

When the variable is AND with 1 and OR with 0, the variable remains the same, i.e.,

$$B.1 = B$$

$$B+0 = B$$

Idempotent Law

When the variable is AND and OR with itself, the variable remains same or unchanged, i.e.,

$$B.B = B$$

$$B+B = B$$

Complement Law

When the variable is AND and OR with its complement, it will give the result 0 and 1 respectively.

$$B.B' = 0 \quad B=1,0$$

$$B+B' = 1$$

Double Negation Law

This law states that, when the variable comes with two negations, the symbol gets removed and the original variable is obtained.

$$((A)')' = A$$

Commutative Law

This law states that no matter in which order we use the variables. It means that the order of variables doesn't matter in this law.

$$A.B = B.A$$

$$A+B = B+A$$

Associative Law

This law states that the operation can be performed in any order when the variables priority is of same as '*' and '/'.

$$(A.B).C = A.(B.C)$$

$$(A+B)+C = A+(B+C)$$

Distributive Law

This law allows us to open up of brackets. Simply, we can open the brackets in the Boolean expressions.

$$A+(B.C) = (A+B).(A+C)$$

$$A.(B+C) = (A.B)+(A.C)$$

Absorption Law

This law allows us for absorbing the similar variables.

$$B+(B.A) = B$$

$$B.(B+A) = B$$

De Morgan Law

The operation of an OR and AND logic circuit will remain same if we invert all the inputs, change operators from AND to OR and OR to AND, and invert the output.

$$(A.B)' = A'+B'$$

$$(A+B)' = A'.B'$$

Proof:

Logic gates

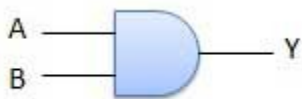
Logic gates are the basic building blocks of any digital system. It is an electronic circuit having one or more than one input and only one output. The relationship between the input and the output is based on a **certain logic**. Based on this, logic gates are named as AND gate, OR gate, NOT gate etc.

AND Gate

A circuit which performs an AND operation is shown in figure. It has n input ($n \geq 2$) and one output.

$$\begin{aligned} Y &= A \text{ AND } B \text{ AND } C \dots\dots N \\ Y &= A.B.C \dots\dots N \\ Y &= ABC \dots\dots N \end{aligned}$$

Logic diagram



Truth Table

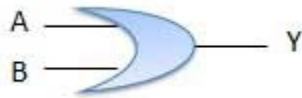
Inputs		Output
A	B	AB
0	0	0
0	1	0
1	0	0
1	1	1

OR Gate

A circuit which performs an OR operation is shown in figure. It has n input ($n \geq 2$) and one output.

$$\begin{aligned} Y &= A \text{ OR } B \text{ OR } C \dots\dots N \\ Y &= A + B + C \dots\dots N \end{aligned}$$

Logic diagram



Truth Table

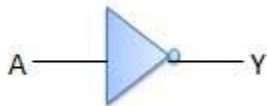
Inputs		Output
A	B	A + B
0	0	0
0	1	1
1	0	1
1	1	1

NOT Gate

NOT gate is also known as **Inverter**. It has one input A and one output Y.

$$\begin{array}{lcl} Y & = & \text{NOT } A \\ Y & = & \overline{A} \end{array}$$

Logic diagram



Truth Table

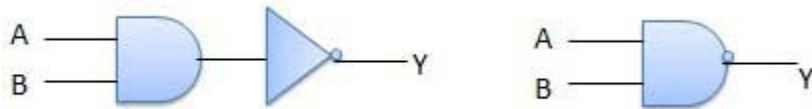
Inputs	Output
A	B
0	1
1	0

NAND Gate

A NOT-AND operation is known as NAND operation. It has n input ($n \geq 2$) and one output.

$$\begin{aligned} Y &= A \text{ NOT AND } B \text{ NOT AND } C \dots\dots N \\ Y &= A \text{ NAND } B \text{ NAND } C \dots\dots N \end{aligned}$$

Logic diagram



Truth Table

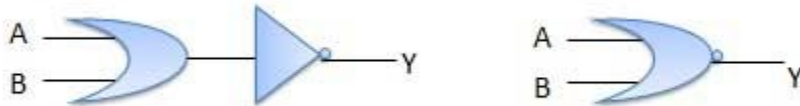
Inputs		Output
A	B	$\overline{AB}$
0	0	1
0	1	1
1	0	1
1	1	0

NOR Gate

A NOT-OR operation is known as NOR operation. It has n input ($n \geq 2$) and one output.

$$\begin{aligned} Y &= A \text{ NOT OR } B \text{ NOT OR } C \dots\dots N \\ Y &= A \text{ NOR } B \text{ NOR } C \dots\dots N \end{aligned}$$

Logic diagram



Truth Table

Inputs		Output
A	B	$\overline{A+B}$
0	0	1
0	1	0
1	0	0
1	1	0

XOR Gate

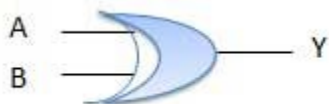
XOR or Ex-OR gate is a special type of gate. It can be used in the half adder, full adder and subtractor. The exclusive-OR gate is abbreviated as EX-OR gate or sometime as X-OR gate. It has n input ($n \geq 2$) and one output.

$$Y = A \text{ XOR } B \text{ XOR } C \dots\dots N$$

$$Y = A \oplus B \oplus C \dots\dots N$$

$$Y = \overline{AB} + \overline{AB}$$

Logic diagram



Truth Table

Inputs		Output
A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

XNOR Gate

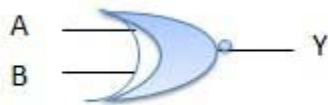
XNOR gate is a special type of gate. It can be used in the half adder, full adder and subtractor. The exclusive-NOR gate is abbreviated as EX-NOR gate or sometime as X-NOR gate. It has n input ($n \geq 2$) and one output.

$$Y = A \text{ XOR } B \text{ XOR } C \dots\dots N$$

$$Y = A \oplus B \oplus C \dots\dots N$$

$$Y = \overline{A B} + A B$$

Logic diagram



Truth Table

Inputs		Output
A	B	$A \oplus B$
0	0	1
0	1	0
1	0	0
1	1	1

Associative Law

Associate Law of Addition

Statement:

Associative law of addition states that OR ing more than two variables i.e. mathematical addition operation performed on variables will return the same value irrespective of the grouping of variables in an equation.

It involves in swapping of variables in groups.

The Associative law using OR operator can be written as

$$A+(B+C) = (A+B)+C$$

Proof:

If A, B and C are three variables, then the grouping of 3 variables with 2 variables in each set will be of 3 types, such as (A + B), (B + C) and (C + A).

According to associative law

$$(A + B + C) = (A + B) + C = A + (B + C) = B + (C + A)$$

We know that, $A + AB = A$ (according to Absorption law)

Now let's assume that, $x = A + (B + C)$ and $y = (A + B) + C$

According to associative law, we need to prove that $x = y$.

Now, find $Ax = A [A + (B + C)]$

$$= AA + A (B + C)$$

$$= A + AB + AC \rightarrow = (A + AB) + AC$$

$$= A + AC \rightarrow \text{since } A + AB = A$$

$$= A \rightarrow \text{since } A + AC = A$$

Therefore $Ax = A$

Similarly, for $Bx = B [A + (B + C)]$

$$= AB + B (B + C)$$

$$= AB + BB + BC$$

$$= AB + B + BC \rightarrow \text{since } BB = B$$

$$= (B + BC) + AB$$

$$= B + AB \rightarrow \text{since } B + BC = B$$

$$= B \rightarrow \text{since } B + AB = B$$

Using these above equations, we can say that the relation between A, B, C and + operator doesn't change when multiplied by other variable like x, such as $xy = yx = x = y$.

$$yx = ((A + B) + C) x$$

$$= (A + B) x + Cx$$

$$= (Ax + Bx) + Cx$$

$$= (A + B) + C$$

$$= y xy = (A + (B + C)) y$$

$$= Ay + (B + C) y$$

$$= Ay + (By + Cy)$$

$$= A + (B + C)$$

$$= x$$

So $x = y$, which means $A + (B + C) = (A + B) + C = B + (A + C)$

Example

Take three variables 0, 1 and 0, then

According to associative law,

$$(0 + 1) + 0 = 0 + (1 + 0)$$

$$1 + 0 = 0 + 1$$

$$1 = 1$$

Hence associative law is verified.

Hence the Associative law is proved, $(A + B + C) = (A + B) + C = A + (B + C) = B + (C + A)$

Associate Law of Multiplication

Statement:

Associative law of multiplication states that ANDing more than two variables i.e. mathematical multiplication operation performed on variables will return the same value irrespective of the grouping of variables in an equation.

The Associative law using AND operator can be written as

$$A * (B * C) = (A * B) * C$$

Distributive law

This is the most used and most important law in Boolean algebra, which involves in 2 operators: AND, OR.

Statement1:

The multiplication of two variables and adding the result with a variable will result in same value as multiplication of addition of the variable with individual variables.

In other words, ANDing two variables and ORing the result with another variable is equal to AND of ORing of the variable with the two individual variables.

Distributive law can be written as

$$A + BC = (A + B)(A + C)$$

This is called OR distributes over AND.

Proof:

If A, B and C are three variables then

$$A + BC = A * 1 + BC \rightarrow \text{since } A * 1 = A$$

$$= A (1 + B) + BC \rightarrow \text{since } 1 + B = 1$$

$$= A * 1 + AB + BC$$

$$= A *(1 + C) + AB + BC \rightarrow \text{since } A * A = A * 1 = A$$

$$= A *(A + C) + B (A + C)$$

$$= (A + C) (A + B)$$

$$A + BC = (A + B) (A + C)$$

Hence, distributive law is proved.

Statement 2:

The addition of two variables and multiplying the result with a variable will result in same value as addition of multiplication of the variable with individual variables.

In other words, ORing two variables and ANDing the result with another variable is equal to OR of ANDing of the variable with the two individual variables.

Distributive law can be written as

$$A (B+C) = (A B) + (A C)$$

This is called AND distributes over OR.

Proof:

$$A (B + C) = A (B*1) + A (C*1) \rightarrow \text{since } 1 * B = B, 1 * C = C$$

$$= [(AB)*(A*1)] + [(AC) *(A*1)]$$

$$=[(AB) * A] + [(AC) *A]$$

$$= (A +1) (AB + AC)$$

$$= (AB +AC) \rightarrow \text{since } 1 + A = 1$$

Hence, distributive law is proved.

Example:

Take three variables 0, 1 and 0, then

According to distributive law,

$$0 (1 + 0) = (0*1) + (0*0)$$

$$0 (1) = (0) + (0)$$

$$0 = 0$$

Hence,distributive law is verified.

Boolean Algebra laws proof

(Distributive law)

$$\begin{aligned}A + BC &= A.1 + BC && [\text{Since, } A.1 = A] \\&= A(1 + B) + BC && [\text{Since, } B+1 = 1] \\&= A.1 + AB + BC \\&= A.(1 + C) + AB + BC \\&= A(A + C) + B(A+C) && [\text{Since, } A.A = A.1 = A] \\A + BC &= (A+B)(A+C)\end{aligned}$$

19

Commutative law

Statement:

Commutative law states that the inter-changing of the order of operands in a Boolean equation does not change its result.

- Using OR operator $\rightarrow A + B = B + A$
- Using AND operator $\rightarrow A * B = B * A$

This law is also has more priority in Boolean algebra.

Example:

Take 2 variables 1 and 0, then

$$1 + 0 = 0 + 1$$

$$1 = 1$$

Similarly,

$$1 * 0 = 0 * 1$$

$$0 = 0$$

Absorption Law

Absorption law involves in linking of a pair of binary operations.

i. $A + AB = A$

ii. $A(A + B) = A$

iii. $A + \bar{A}B = A + B$

iv. $A(\bar{A} + B) = AB$

3rd and 4th laws are also called as Redundancy laws.

Statement 1: $A + AB = A$

Proof:

$$A + AB = A.1 + AB \rightarrow \text{since } A.1 = A$$

$$= A(1+B) \rightarrow \text{since } 1 + B = 1$$

$$= A.1$$

$$= A$$

Statement 2: $A(A + B) = A$

Proof:

$$A(A + B) = A.A + A.B$$

$$= A + AB \rightarrow \text{since } A . A = A$$

$$= A(1 + B)$$

$$= A.1$$

$$= A$$

Statement 3: $A + \bar{A}B = A + B$

Proof:

$A + \bar{A}B = (A + \bar{A})(A + B) \rightarrow$ since $A+BC = (A+B)(A+C)$ using distributive law

$= 1 * (A + B) \rightarrow$ since $A + \bar{A} = 1$

$= A + B$

Statement 4: $A * (\bar{A} + B) = AB$

Proof: $A * (\bar{A} + B) = A. \bar{A} + AB$

$= AB \rightarrow$ since $A \bar{A} = 0$

De Morgan's Theorem

Boolean algebra involves in binary addition, binary subtraction, binary division and binary multiplication of binary numbers. Similar to these basic laws, there is another important theorem in which the Boolean algebraic system mostly depends on. That is De Morgan's law.

This can be also known as De Morgan's theorem. This law works depending of the concept of Duality. Duality means interchanging the operators and variables in a function, such as replacing 0 with 1 and 1 with 0, AND operator with OR operator and OR operator with AND operator.

De Morgan's law is like extension of the Duality principle. De Morgan proposed 2 theorems, which will help us in solving the algebraic problems in digital electronics.

The De Morgan's statements are,

Statement 1:

“The negation of conjunction is the disjunction of the negations”. Or we can define that as “The compliment of the product of 2 variables is equal to the sum of the compliments of individual variables”.

$$(A.B)' = A' + B'$$

Statement 2:

“The negation of disjunction is the conjunction of the negations”. Or we can define that as “The compliment of the sum of two variables is equal to the product of the compliment of each variable”.

$$(A + B)' = A'.B'$$

Truth Tables

The De Morgan’s laws are simply explained by using the truth tables.

The truth table for De Morgan’s first statement $((A.B)' = A' + B')$ is given below.

A	B	A'	B'	A.B	(A.B)'	A'+B'
0	0	1	1	0	1	1
0	1	1	0	0	1	1
1	0	0	1	0	1	1
1	1	0	0	1	0	0

A	B	C	A'	B'	C'
0	0	0			
0	0	1			
0	1	0			
0	1	1			
1	0	0			
1	0	1			
1	1	0			
1	1	1			

So the De Morgan's first law can also be expressed as “not (A and B) is equal to (not A) or (not B)”.

The truth table for De Morgan's second statement $((A + B)' = A'.B')$ is given below.

A	B	A'	B'	A+B	$(A+B)'$	$A'.B'$
0	0	1	1	0	1	1
0	1	1	0	1	0	0
1	0	0	1	1	0	0
1	1	0	0	1	0	0

So the De Morgan's first law can also be expressed as “not (A or B) is equal to (not A) and (not B)”.

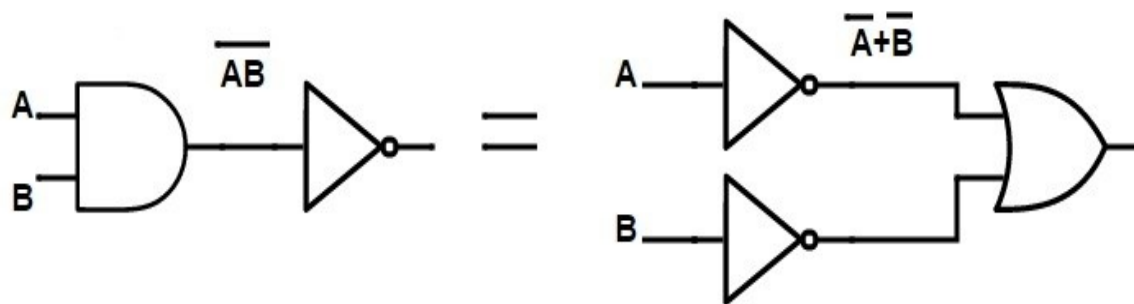
DeMorgan's Theorem in Gates

The De Morgan's theorem can be proved by using basic logic gates such as AND gates and OR gates.

For statement 1: $(A.B)' = A' + B'$

The output of a NAND gate (AND gate with a NOT gate at its output side) is equal to output of the gate formed by connecting two NOT gates at the inputs of OR gate. This can be said as,

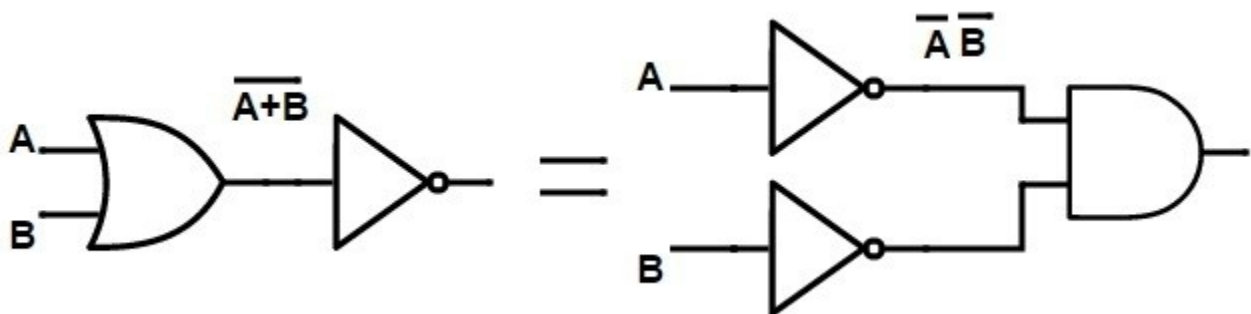
NAND gate = Bubbled OR gate



For statement 2: $(A + B)' = A'.B'$

The output of a NOR gate (OR gate with a NOT gate at its output side) is equal to output of the gate formed by connecting two NOT gates at the inputs of AND gate. This can be said as,

NOR gate = Bubbled AND gate

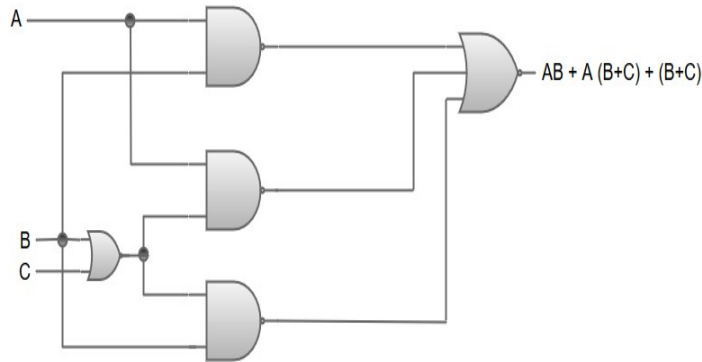


Let's see some examples to understand how the De Morgan's theorem is used to simplify the Boolean equations.

Simplification using Boolean algebra

Let us consider an example of a Boolean function: $AB + A(B+C) + B(B+C)$

The logic diagram for the Boolean function $AB + A(B+C) + B(B+C)$ can be represented as:



We will simplify this Boolean function on the basis of rules given by Boolean algebra.

$$AB + A(B+C) + B(B+C)$$

$$AB + AB + AC + BB + BC \quad \{\text{Distributive law; } A(B+C) = AB+AC, B(B+C) = BB+BC\}$$

$$AB + AB + AC + B + BC \quad \{\text{Idempotent law; } BB = B\}$$

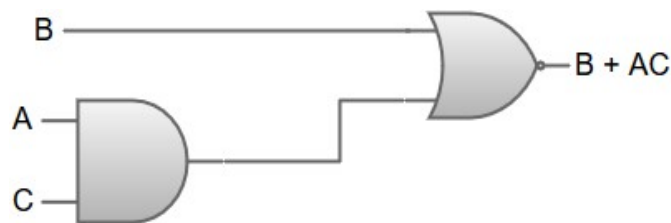
$$AB + AC + B + BC \quad \{\text{Idempotent law; } AB+AB = AB\}$$

$$AB + AC + B \quad \{\text{Absorption law; } B+BC = B\}$$

$$B + AC \quad \{\text{Absorption law; } AB+B = B\}$$

Hence, the simplified Boolean function will be $B + AC$.

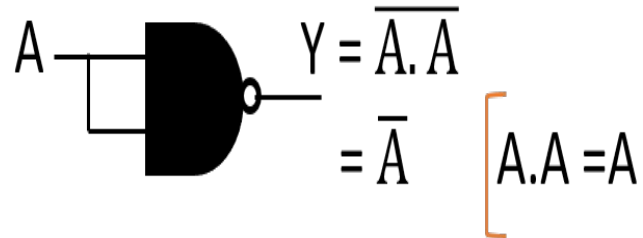
The logic diagram for Boolean function $B + AC$ can be represented as:



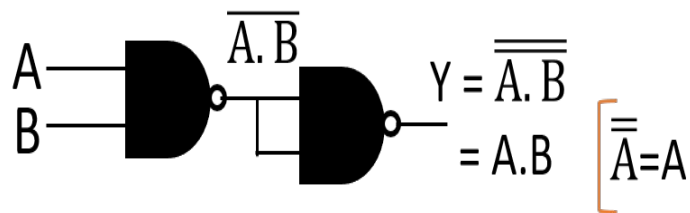
Proof of Universality of NAND and NOR Gate

Proof of Universality of NAND Gate:

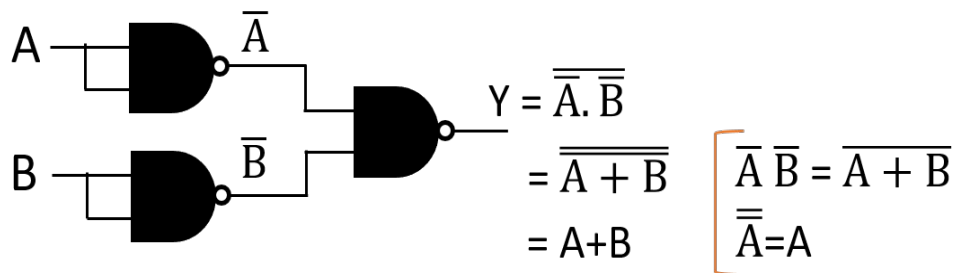
Implementation of NOT gate using only NAND gate:



Implementation of AND gate using only NAND gate:



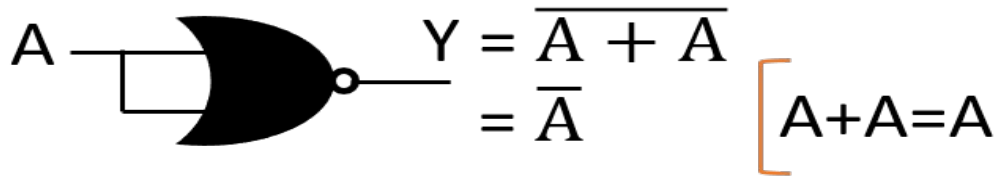
Implementation of OR gate using only NAND gate:



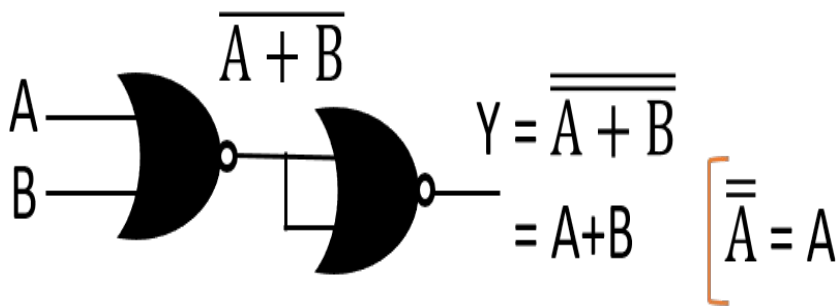
As we can see from the above discussion, All three basic gates can be implemented by only NAND Gate. Again we know that any gate or any circuit can be implemented by three basic gates. Since NAND Gate can implement any gate or any circuit with three basic gates (AND, OR, NOT), NAND Gate is called a universal Gate.

Proof of Universality of NOR Gate:

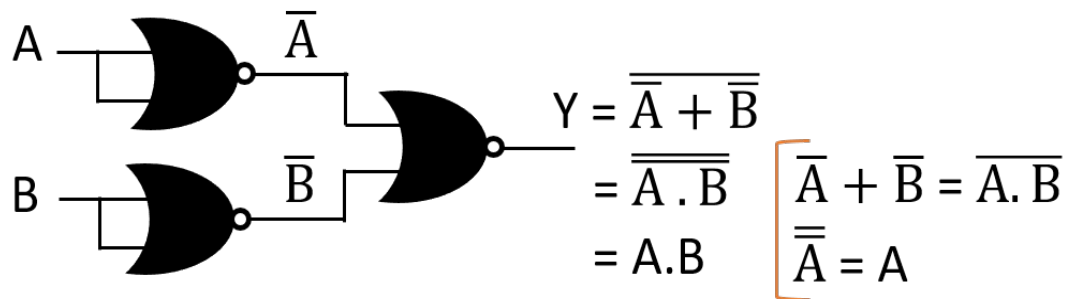
Implementation of NOT gate using only NOR gate:



Implementation of OR gate using only NOR gate:



Implementation of AND gate using only NOR gate:

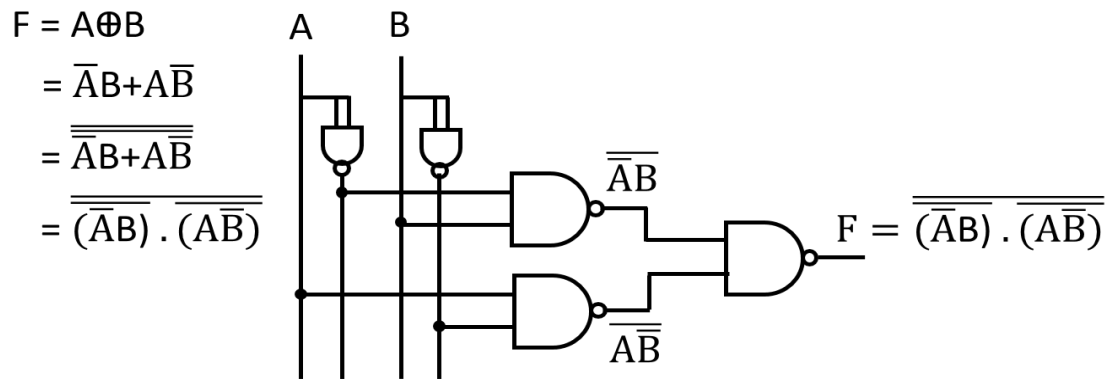


As we can see from the above discussion, All three basic gates can be implemented by only NOR Gate. Again we know that any gate or any circuit can be implemented by three basic gates. Since NOR Gate can implement any gate or any circuit with three basic gates (AND, OR, NOT), NOR Gate is called a universal Gate.

[When implementing any function using only NAND gate, you should see if the function has any OR operator? If so, convert the OR to AND via the De Morgan's theorem and then implement.]

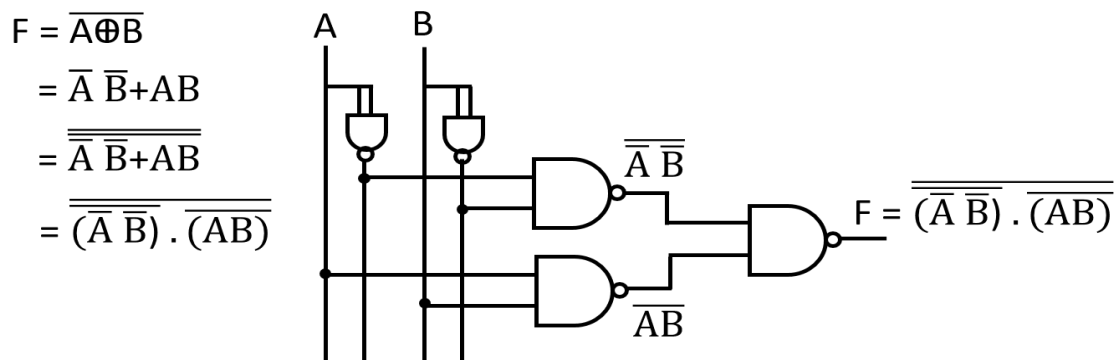
Implementation of 2-input XOR gate using only NAND gate:

Boolean function of the XOR gate for variables A and B,



Implementation of 2-input XNOR gate using only NAND gate:

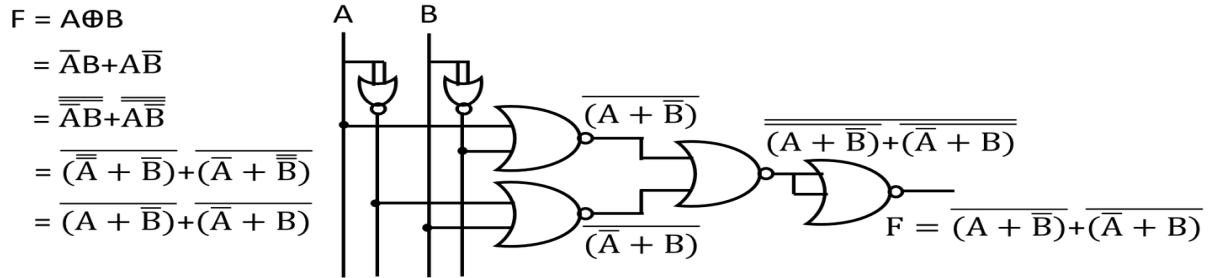
Boolean function of the XNOR gate for variables A and B,



[When implementing any function using only NOR gate, you should see if the function has any AND operator? If so, convert the AND to OR via the De Morgan's theorem and then implement.]

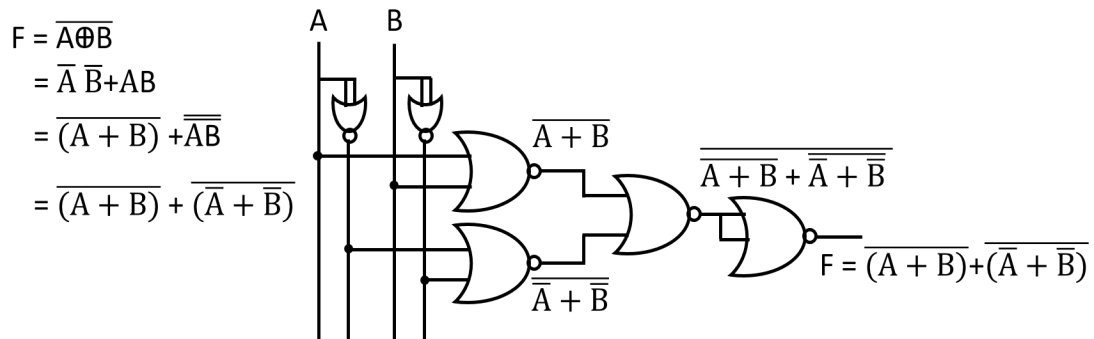
Implementation of 2-input XOR gate using only NOR gate:

Boolean function of the XOR gate for variables A and B,



Implementation of 2-input XNOR gate using only NOR gate:

Boolean function of the XNOR gate for variables A and B,



Simplify: $C + BC$:

<u>Expression</u>	<u>Rule(s) Used</u>
$C + BC$	Original Expression
$C + (B + C)$	DeMorgan's Law.
$(C + C) + B$	Commutative, Associative Laws.
$T + B$	Complement Law.
T	Identity Law.

$$Y = AB + AB$$

A	B	C	output		
0	0	0	1	$A'B'C'$	
0	0	1	1	$A'B'C$	
0	1	0	1	$A'BC'$	
0	1	1	0		$Y = A'B'C' + A'B'C + A'BC' + AB'C'$
1	0	0	1	$AB'C'$	
1	0	1	0		
1	1	0	0		
1	1	1	0		

A. B(product)

A+B(Sum)

Sum of product(SOP)

Product of Sum(POS)